Interactive AI Museums — Design Spec

> ***Status:*** *design-stage. To be reviewed section-by-section, then handed to* `superpowers:writing-plans` *for an implementation plan.*
>
> ***Project mode:*** *portfolio project. Ship for colleague review and a public launch; revisit strategy after.*

---

§0 — One-page summary

A web product where the user types a one-line prompt (*"a Victorian study with a working pocket watch," "an exhibit on deep-sea creatures," "a museum of Renaissance inventions"*) and gets back a **3D exhibit** they can move through and explore. The exhibit consists of:

- an **AI-generated environment** (a single room or hall, generated by HunyuanWorld; or a curated stock room as fallback);
- **three to five AI-generated objects** placed on pedestals, generated individually by Microsoft TRELLIS.2 with internal structure;
- a **camera that moves along a Bezier spline** through the environment, advancing station-by-station — no free walking, no collision system;
- a **dissection viewer** that lets users cross-section any object with a clipping plane and (where applicable) view its components in an exploded layout.

Three curated launch exhibits — **Anatomy** (heart, brain, skeleton), **Horology** (three pocket watches), **Botany** (fruits, seeds, plant specimens) — plus an open prompt box for user submissions.

The product's strategic position: not competing with WorldLabs Marble on world-generation quality, but sitting *next to it* in a different category — **interactivity and dissection**, not generation. Marble outputs are *assets you download*. Ours are *places you visit*.

Six-week build, gated on a **three-day spike in week 1** that validates the three things that could kill the demo (TRELLIS.2 interior structures, HunyuanWorld walkability under camera motion, three.js capped-clipping shader on real TRELLIS.2 GLBs).

---

§1 — Context, goals, non-goals

Why this project

In the past year, generative 3D-scene tools (WorldLabs Marble; HunyuanWorld; LucidDreamer; Director3D) have made it possible to produce a walkable 3D environment from a text prompt. But every one of these tools produces **dead worlds** — you can look around, but you can't pick anything up, open anything, or examine anything. The objects in these worlds are scenery, not things.

This project fills that gap. It does *not* try to beat Marble at scene-generation quality (Marble has $230M in funding and a year's head start). It uses comparable open-source scene-generation models as **substrate** and concentrates engineering effort on the **interaction layer** — the part nobody else is building.

Goals (v1)

1. Type a prompt → get an interactive 3D exhibit in a browser, no installs, in under three minutes.
2. Three curated launch exhibits that look polished enough to anchor the launch (Anatomy, Horology, Botany).
3. An open prompt box that produces "good enough" exhibits for the long tail of user prompts.

4. A single shareable URL per exhibit. Recipients land directly in the same experience.
5. Cross-section / exploded-view dissection on objects with internal structure.
6. Ship within six weeks of week-1 spike pass.

Non-goals (v1)

- True free walking with collision detection inside the AI scene.
- Object physics (pick up, throw, drop).
- Audio, ambient music, voice narration.
- Multi-user — no shared sessions, no presence.
- Mobile responsiveness for the viewer (landing page is mobile-friendly; the viewer is desktop-only).
- Editing primitives — no scene-modification, no object-replacement, no regenerate-this.
- Auth, accounts, payments, rate limits beyond a global per-IP cap.
- Educational content packaging (lesson plans, classroom mode) — that's a possible v2.
- Production-grade content moderation. v1 has a prompt-level Claude refusal filter; output-level moderation is a v2 concern.

Strategic constraint

Portfolio mode. Optimize for:

- a polished launch demo that screenshots and video-clips well;
- a clear "interactive museum" framing that colleagues immediately understand;
- low recurring cost (< $50/month at sane traffic) so it can stay online.

Not optimizing for:

- scale beyond ~1K exhibits / month;
- multi-tenant operation;
- a v2 commercial story (we'll figure that out after launch feedback).

---

§2 — Locked decisions

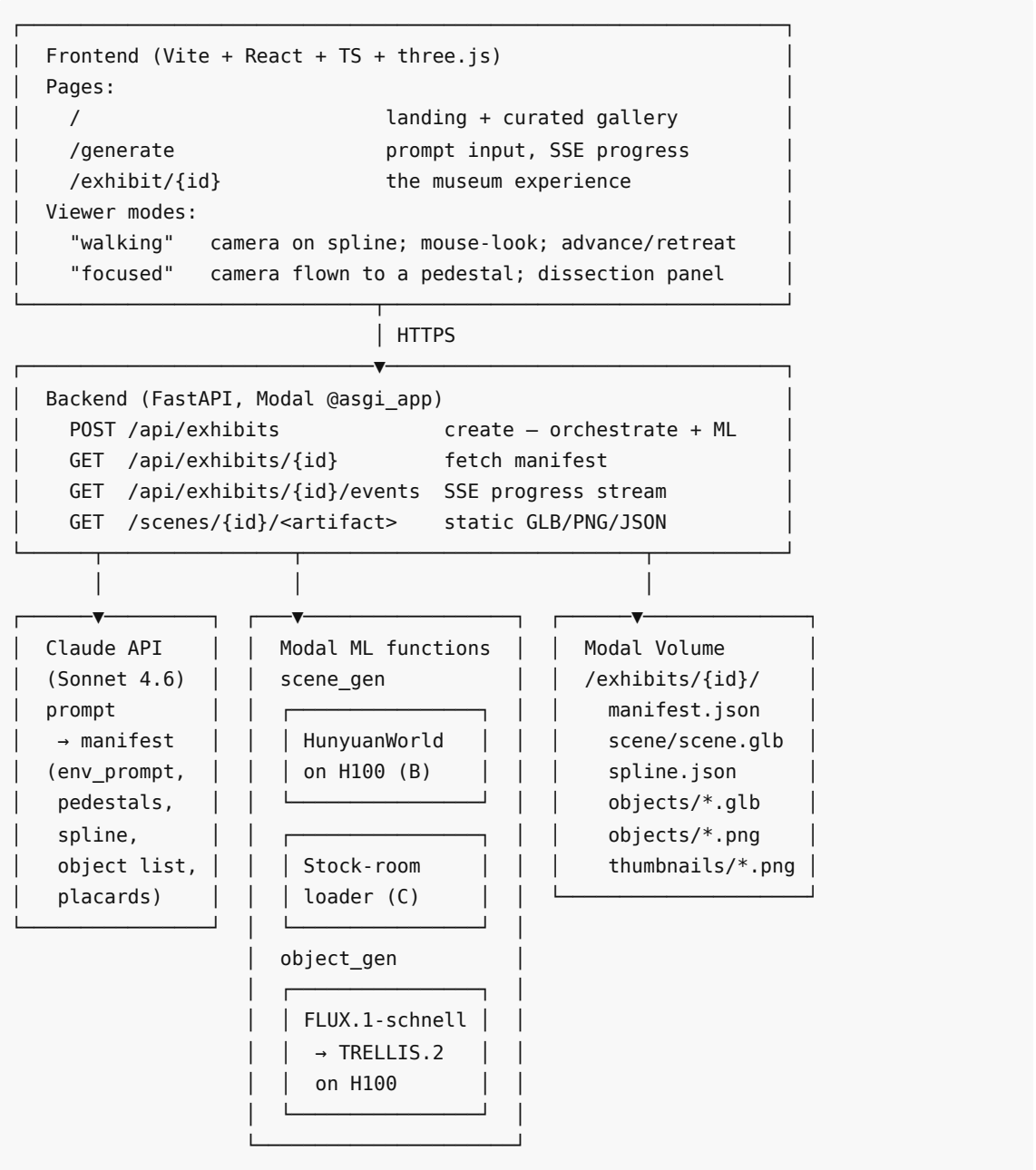
1. **Project mode:** portfolio project. Ship, share with colleagues, revisit strategy after.
2. **Repository root:** `/mnt/3D_Mapping/`. Existing scaffolding (PLAN_v4, references/) carried forward.
3. **Frontend:** Vite + React 19 + TypeScript + three.js (not React Three Fiber — we need direct shader control for capped clipping).
4. **Backend:** FastAPI + Pydantic v2, deployed as a Modal `@asgi_app`. No Celery. No Redis. No SQLite. No SSH adapter. Modal's built-in queue suffices.
5. **Persistence:** per-exhibit `manifest.json` on Modal Volume. No relational DB. Manifest is the single source of truth.
6. **Orchestrator:** one Claude API call per exhibit (Sonnet 4.6). Generates the full manifest including scene prompt, object list with image prompts, spline, and pedestal layout.
7. **Scene generator:** HunyuanWorld on H100 via Modal (Approach B). Fallback to curated stock rooms (Approach C) if HunyuanWorld fails the spike or fails at runtime.
8. **Object generator:** FLUX.1-schnell (image) → TRELLIS.2 (mesh+PBR). Both Apache-2.0, both on Modal H100.
9. **Locomotion:** camera-on-rails along a Catmull-Rom spline. Station-by-station advance/retreat. Free mouse-look. No collision. No free walking.
10. **Object interaction:** camera flies to a hero pose at the object (focus mode). Object stays on its pedestal. Mouse drag rotates the object in place. Side panel exposes dissection controls.
11. **Canonical artifact format:** GLB + PBR (TRELLIS.2 native). No Gaussian splats — the entire v3 "splat as canonical" framing is dropped.

12. **No mock outputs.** Stages that aren't implemented yet return 501 with the stage name. Faking outputs hides wiring bugs.
13. **Launch content:** three curated exhibits (Anatomy, Horology, Botany) + one open prompt box. Cut Botany to week 7 if scope slips.
14. **Hosting:** frontend on Cloudflare Pages (free tier). Backend + ML on Modal. Domain TBD.

---

§3 — System architecture

One web app, one backend, one ML host, one orchestrator, one storage layer.



What's not in the diagram (intentional)

- No Celery / Redis. Modal's `.spawn()` queue handles async ML calls.

- No SQLite / Postgres. Manifest on volume is enough for v1's read patterns.
- No SSH adapter to an external H100. Modal hosts the GPU.
- No CDN — Modal's static fileserver is enough at v1 traffic.
- No multi-region — single Modal region.

---

§4 — Orchestrator and manifest schema

The orchestrator is one Claude API call. It is the planning layer of the system. Every downstream component reads from its output. The system prompt is the highest-value piece of IP in the project.

Manifest schema (canonical contract)

```
{
  "schema_version": "v4.1",
  "exhibit_id": "01HZ8X9A2B...",
  "created_at": "2026-05-13T18:00:00Z",
  "user_prompt": "A Victorian study with a working pocket watch",
  "title": "The Inventor's Study",
  "description": "An evening in a 19th-century inventor's private room.",

  "scene": {
    "generator": "hunyuanworld",
    "scene_prompt": "interior of a Victorian study, oak panelling, brass fixtures,
warm afternoon light through a tall window, no people, no foreground objects, photo-
real",
    "stock_room_id": null,
    "scene_url": "/exhibits/{id}/scene/scene.glb",
    "ambient_light": { "color": "#f4e2c4", "intensity": 0.6 },
    "fog": { "color": "#1a1208", "near": 8, "far": 25 }
  },

  "spline": {
    "control_points": [
      [0.0, 1.7, 4.0],
      [-1.0, 1.7, 2.5],
      [-1.0, 1.7, 0.0],
      [1.5, 1.7, -1.0]
    ],
    "stations": [
      { "t": 0.00, "label": "entrance",   "pedestal_id": null,
"preferred_yaw_deg":   0 },
      { "t": 0.33, "label": "the watch",  "pedestal_id": "pedestal_1",
"preferred_yaw_deg": -90 },
      { "t": 0.66, "label": "the books",  "pedestal_id": "pedestal_2",
"preferred_yaw_deg":   0 },
      { "t": 1.00, "label": "the candle", "pedestal_id": "pedestal_3",
"preferred_yaw_deg":  90 }
    ],
    "travel_time_s": 6.0
  },
```

```
  "pedestals": [
    { "id": "pedestal_1", "position": [-1.0, 0.0,  0.5], "height": 0.95,
"object_id": "watch_1"  },
    { "id": "pedestal_2", "position": [-1.0, 0.0, -0.5], "height": 0.95,
"object_id": "book_1"   },
    { "id": "pedestal_3", "position": [ 1.5, 0.0, -1.0], "height": 0.95,
"object_id": "candle_1" }
  ],

  "objects": [
    {
      "id": "watch_1",
      "label": "Antique pocket watch",
      "placard_description": "An open-faced pocket watch, c. 1880, brass case,
exposed escapement and mainspring. Drag the slider to see the gear train inside.",
      "image_prompt": "an antique brass pocket watch, back cover removed, gears and
balance wheel exposed, hero shot, plain white background, centered, photographic
quality, single subject, no text",
      "image_url": "/exhibits/{id}/objects/watch_1.png",
      "glb_url":   "/exhibits/{id}/objects/watch_1.glb",
      "rotation_y_deg": 30,
      "scale": 0.08,
      "dissectable": true,
      "preferred_clip_axis": "y",
      "preferred_clip_t": 0.5
    }
    /* book_1, candle_1 ... */
  ],

  "stage": "ready",
  "timing": {
    "orchestrator_ms": 1850,
    "scene_gen_ms": 240000,
    "objects_gen_ms": { "watch_1": 31200, "book_1": 29400, "candle_1": 22100 },
    "total_ms": 327050
  },
  "errors": []
}
```

Orchestrator system prompt (sketch)

Lives in `backend/app/orchestrator/prompts.py` as a versioned constant with embedded few-shot examples. The prompt has six jobs, in order:

1. **Refuse harmful prompts.** NSFW, real-person likeness, gore-with-realism, hate content → return `{ "refused": true, "reason": "..." }` with a short, non-lecturing refusal. Backend returns 400 to the user.
2. **Pick 3–5 objects.** Each must be a discrete, isolatable, hero-shot-able subject (a thing, not a setting attribute). Bias toward objects with plausible internal structure for the dissection moment (mechanical, anatomical, layered, food with interior).
3. **Write the `scene_prompt`** for HunyuanWorld. Describes ambient features only (architecture, materials, lighting, mood). **Explicitly excludes** foreground objects — those are generated separately

and placed as GLB pedestals. Boilerplate ending: `"no people, no foreground objects, empty pedestals at center-room."`

4. **Write per-object `image_prompt`s** for FLUX.1-schnell. Fixed scaffold: `"<thing>, hero shot, plain white background, centered, photographic quality, single subject, no text"` plus a dissection hint if relevant (`"cross-sectioned"` , `"back cover removed"` , `"sliced open"`).
5. **Lay out spline + pedestals.** Pick a layout template based on object count: *linear gallery* (3 objects, side-by-side), *hub-and-spokes* (4–5 objects, central pivot), *semicircle* (any count). Eye height = 1.7m; stations spaced for 5–8s travel; control points produce a curve that doesn't pass through any pedestal.
6. **Write `title` (≤4 words) + `description` (1–2 sentences)** for the share card and landing thumbnail.

Output is strict JSON conforming to the manifest schema. Pydantic validates server-side. On validation failure, re-prompt once with the validation error inlined. On second failure, return 502 to the user with "Could not plan this exhibit. Try a different prompt." Refused prompts return 400.

Cost

Sonnet 4.6 at ~3K input tokens (system prompt + 2 few-shot examples) + ~2K output tokens (manifest) ≈ **$0.02 per exhibit**. Negligible vs. ML cost.

Why Claude API rather than self-host

Orchestrator quality dominates exhibit quality. Sonnet's structured-output reliability, instruction-following, and layout-reasoning are above anything self-hostable at similar latency. $0.02 per exhibit is not a bottleneck.

---

§5 — Rails locomotion

Two camera modes; transitions on user input.

Walking mode (default)

Camera position interpolates along the manifest's spline. Input grammar:

Input	Action
Space / → / ↓	advance to next station
← / ↑	retreat to previous station
mouse (pointer-locked)	free yaw + pitch at all times
click pedestal	enter focus mode for that object
Esc	release pointer lock; show overlay

The spline is **Catmull-Rom** through `manifest.spline.control_points` — interpolates exactly through each point, no tangent spec needed. `spline.position(t)` and `spline.tangent(t)` are evaluated each frame.

Stations are discrete `t` values from `manifest.spline.stations` . Client state holds `current_station_index` . Advancing tweens the camera from current `t` to next station's `t` over

`manifest.spline.travel_time_s` (default 6s) with ease-in-out. On arrival, the camera smoothly yaws to `preferred_yaw_deg` over ~0.8s. After arrival, mouse-look has full control.

Mouse-look during a transition is **additive** — the tween drives base orientation; mouse delta layers on top. The user can look around while moving without fighting the camera.

If the user presses advance while already mid-transition, **interrupt and re-target** to the next-next station. Keeps controls responsive.

Focused mode

Click a pedestal → camera animates from spline to a **hero pose**:

- Position: along the camera's forward axis, 0.6m from object center (object fills ~⅓ of frame).
- Look-at: object's bounding-box centroid.
- Animation: ~1.0s ease-in-out.

In focused mode:

- Mouse drag rotates the **object** (not the camera) around its own axes.
- Scroll dollies camera in/out (bounded).
- Right side panel reveals dissection controls (see §6).
- `Esc` or "Back to exhibit" button: tween back to spline at last walking-mode `t`.

We do **not** physically pick up the object. Camera-to-object is the consistent gesture given rails locomotion — if the user isn't walking, the object shouldn't be moving either.

HUD in walking mode

Minimal:

- Top center: `Station 2 of 4 — The Watch` (current station label).
- Bottom: thin spline-progress bar.
- Bottom-right hint chip: `→ next ← back click to inspect` (fades after first input).
- No crosshair, no reticle. It's a museum.

Modules

```
frontend/src/viewer/locomotion/
├── Spline.ts            Catmull-Rom: position(t), tangent(t), arcLength()
├── RailController.ts    advance/retreat, tween + yaw-align
├── FocusController.ts   enter/exit focus; hero-pose calc; object-space drag
├── MouseLook.ts         pointer-lock, yaw/pitch delta, additive blend
└── ModeManager.ts       state machine: walking ↔ focused
```

State in Zustand (`useLocomotionStore`). All animation in a single `useFrame` callback at the viewer root.

Edge cases

- **Pointer lock lost** (tab switch, Escape): freeze camera, show "Click to resume" overlay. Resume on click.
- **Manifest < 2 stations:** invalid — rejected server-side by Pydantic before manifest is returned.
- **Pedestal click during transition:** interrupt walking tween, jump straight into focus mode.
- **Mobile or low-end browser:** show "best on desktop" notice with link to the curated video gallery. Don't render the viewer.

---

§6 — Dissection viewer (the moat)

The technical centerpiece of the project. Three sub-features in the focused-mode side panel.

6.1 Clipping plane

three.js native via `material.clippingPlanes`. One movable `THREE.Plane` per focused object, anchored to object-local origin. User controls a slider `t ∈ [0, 1]` mapped to the plane's offset across the object's local bounding box along the chosen axis.

Axis dropdown (X/Y/Z). Initial axis = `manifest.objects[i].preferred_clip_axis`. Initial slider = `manifest.objects[i].preferred_clip_t` — gives the user a sensible first cut without fiddling.

6.2 Capped cross-section (the engineering challenge)

Without capping, clipped objects look hollow — the slice reveals a hole rather than a solid interior face. We need to fill the cut surface with a color sampled from the material's albedo.

Approach (stencil buffer + cap pass):

1. Pass 1: render scene with `clippingPlanes` enabled and `material.stencilWrite = true, stencilRef = 1`. The stencil buffer marks pixels where the clip occurred.
2. Pass 2: render a full-screen quad coplanar with the clip plane, masked by the stencil. Fragment shader samples the per-object albedo (uniform from the material) and writes that as the cap color.
3. Optional: a thin emissive rim at the cut edge for visibility.

Reference implementation: three.js example `webgl_clipping_stencil.html` — adapt for our material setup and dynamic clip-axis selection.

Fallback if too finicky: ship v1 *without* capping. Clipped objects look hollow, but the interior content (gears, seeds, chambers) is the star — the cap is polish. Mark capping as v2 polish, ship the dissection without it. Spike C tests this in week 1.

6.3 Exploded view

TRELLIS.2 sometimes emits hierarchical GLBs with multiple meshes (e.g., the watch case + the gear train + the back cover as separate meshes). For multi-mesh objects:

1. On GLB load, count `gltf.scene.children`. If > 1, exploded view is available.
2. Compute each child mesh's centroid.
3. At slider `t ∈ [0, 1]`, translate each mesh outward from the object's overall center by `t * k * |centroid - object_center|` where `k ≈ 1.5`.

Single-mesh objects: exploded slider disabled with hint text "Single component — no parts to separate."

6.4 Focus-mode side panel

Right side, ~320px wide. Card style. Contents top to bottom:

```
──────────────────────────────────
  Antique pocket watch
──────────────────────────────────
  An open-faced pocket watch,
  c. 1880, brass case, exposed
  escapement and mainspring.
```

```
──────────────────────────────────

  Cut along:  [Y ▾]

  Cross-section position
  ▶──────●──────────  47%

  Exploded view
  ▶──────────────────  0%

  [ Reset ]    [ ↶ Back to exhibit ]
──────────────────────────────────
```

Sliders debounced (~50ms) and animated, so dragging produces smooth visual change rather than jitter.

6.5 Performance

- Single object in focused mode → trivial GPU cost.
- Stencil + cap pass adds one extra render → fine at 60fps on mid-range hardware.
- TRELLIS.2 outputs typically <50K triangles per object with one albedo + roughness + metallic + normal texture set. Well within budget.

6.6 Modules

```
frontend/src/viewer/dissection/
├── ClippingPlane.ts       plane math; axis-aware offset
├── CappedClipMaterial.ts  extends MeshStandardMaterial; stencil + cap-pass logic
├── ExplodedView.ts        per-mesh centroid extraction + radial translation
├── DissectionPanel.tsx    React side panel (sliders, axis dropdown, buttons)
└── DissectionStore.ts     Zustand: { focusedObjectId, clipAxis, clipT, explodeT }
```

---

§7 — Scene generation pipeline + fallback

Primary: HunyuanWorld (Approach B)

Tencent open-source, MIT license. Generates a 3D scene (Gaussian splat or hybrid mesh) from a text prompt. Hosted as a Modal `@modal.function(gpu="H100", image=hunyuan_image)`. Expected latency: 3–5 minutes per scene at decent quality.

Input: `scene_prompt` from manifest. Output: `scene.glb` (or `.ply` if splat-only — TBD per spike). Written to `/exhibits/{id}/scene/`.

Validation

After generation, automated checks:

- File present and parseable.
- Mesh/splat has nonzero geometry within sensible bounding box.
- Test-render from a random spline position: not all black, not all-noise.

Failed check → fallback to stock rooms.

Fallback: stock rooms (Approach C)

Curated library of ~10 pre-made room environments, stored as GLB in `backend/assets/stock_rooms/`. Each tagged with descriptive metadata (`{"id": "victorian_study_001", "themes": ["historical", "study", "warm-light"], "style": "photoreal"}`).

When fallback triggers, the orchestrator is re-invoked with a `force_generator="stock"` directive. The system prompt instructs it to pick the best-matching `stock_room_id` from a provided list. Manifest is re-emitted with `scene.generator = "stock"`, `scene.stock_room_id = "..."`, and an unchanged spline (the orchestrator generates spline/pedestals in coordinates consistent with the stock room's known geometry).

User-facing: SSE event `scene_fallback` with the original scene-prompt and the reason. Frontend shows a small flag in the exhibit footer and an optional "Try AI scene again" button (retries HunyuanWorld with a different seed).

Stock-room sourcing

For launch, source CC0 room GLBs from Polyhaven, Sketchfab CC0 collection, and bespoke Blender renders. ~10 rooms covering: study, library, workshop, gallery, classroom, laboratory, kitchen, conservatory, attic, outdoor pavilion. Each ~5–20 MB.

---

§8 — Object generation pipeline

Per object: FLUX.1-schnell → TRELLIS.2. Both on Modal H100.

Step 1: FLUX.1-schnell (text → image)

Apache-2.0 weights. 4 inference steps. ~2s on H100. Produces a 1024×1024 PNG conforming to `image_prompt` (clean background, hero shot, single subject).

Latency dominated by H100 cold-start. With warm function: ~3s end-to-end including I/O.

Step 2: TRELLIS.2 (image → mesh+PBR)

Microsoft, MIT license. 4B parameters. Input: 1024×1024 image. Output: GLB with PBR textures (base color, roughness, metallic, opacity) at resolution 512^3 (~~30s) or up to 1536^3 (~~60s on H100).

We use **512^3** by default for launch speed; 1024^3 for the three curated exhibits (worth the extra ~30s per object).

Health checks (post-generation)

- GLB parseable; non-empty mesh.
- Vertex count in [1K, 500K].
- Bounding box volume > 0 and finite.
- Texture maps present and decodable.

Failed check → mark this object failed in manifest; exhibit ships with remaining objects.

Parallelism

The orchestrator emits N objects (3–5). The backend dispatches all N `object_gen` calls in parallel via `asyncio.gather` against Modal `.spawn()`. Wall-clock time for all objects ≈ wall-clock time for the slowest one.

Cost

Per object on Modal H100: ~~30s × $0.0035/s = **~~$0.10** at 512³, or ~$0.20 at 1024³. Five objects × $0.10 = **$0.50** per exhibit at the default tier.

---

§9 — Module layout

```
/mnt/3D_Mapping/
├── README.md
├── docs/superpowers/specs/
│   └── 2026-05-14-interactive-museums-design.md     this spec
├── docs/superpowers/plans/                          (created by writing-plans)
├── SPIKE_RESULT.md                                  populated end of week 1
├── pyproject.toml                                   uv workspace root
├── Makefile                                          make dev | spike | deploy
├── .env.example
├── backend/
│   ├── pyproject.toml
│   └── app/
│       ├── main.py                                  FastAPI + Modal asgi_app
│       ├── config.py                                pydantic-settings
│       ├── api/
│       │   ├── exhibits.py                          POST/GET/SSE
│       │   └── health.py
│       ├── schemas.py                               Manifest, Scene, Object, Spline,
Pedestal
│       ├── orchestrator/
│       │   ├── claude.py                            Claude API client
│       │   ├── prompts.py                           versioned system prompts + few-
shot
│       │   └── layout_templates.py                  linear, hub-and-spokes,
semicircle
│       ├── modal_client/
│       │   ├── scene.py                             HunyuanWorld + stock loader
│       │   ├── object.py                            FLUX → TRELLIS.2 chain
│       │   └── _modal.py                            Modal app + image setup
│       ├── stock_rooms/                             metadata + GLB paths
│       │   ├── manifest.json
│       │   └── rooms/*.glb
│       └── storage/
│           └── volume.py                            Modal volume read/write
├── modal/                                           Modal function definitions
│   ├── scene_gen.py
│   ├── object_gen.py
│   └── shared.py
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── api/client.ts                            typed API wrappers
│       ├── pages/
```

```
│       │   ├── Landing.tsx
│       │   ├── Generate.tsx                          prompt + SSE progress
│       │   └── Exhibit.tsx                           the viewer host
│       ├── viewer/
│       │   ├── ExhibitViewer.tsx                     three.js root
│       │   ├── Scene.tsx                             scene GLB loader
│       │   ├── Pedestals.tsx                         pedestal meshes + object
placement
│       │   ├── locomotion/
│       │   │   ├── Spline.ts
│       │   │   ├── RailController.ts
│       │   │   ├── FocusController.ts
│       │   │   ├── MouseLook.ts
│       │   │   └── ModeManager.ts
│       │   ├── dissection/
│       │   │   ├── ClippingPlane.ts
│       │   │   ├── CappedClipMaterial.ts
│       │   │   ├── ExplodedView.ts
│       │   │   ├── DissectionPanel.tsx
│       │   │   └── DissectionStore.ts
│       │   └── hud/
│       │       ├── StationIndicator.tsx
│       │       ├── ProgressBar.tsx
│       │       └── HintChip.tsx
│       └── components/                               landing-page bits
└── references/                                       gitignored
    ├── TRELLIS.2/                                     cloned in T2
    ├── HunyuanWorld/                                  cloned in T2
    ├── supersplat/                                    for editor reference
    └── existing 3DGS clones (relocated from v3)
```

---

§10 — Spike plan (Week 1)

Three concrete proofs. Each is one day. Each has a binary go/no-go. **All three must pass before week 2 starts.** Adapt paths defined per spike.

Spike A — TRELLIS.2 interiors hold up (Day 1)

Generate hero images via FLUX.1-schnell + run TRELLIS.2 on these 10 prompts:

1. a pomegranate cross-sectioned, seeds visible
2. a vintage mechanical pocket watch, back cover removed, gears exposed
3. a human heart, chambers visible
4. an apple sliced in half, core visible
5. a Gothic cathedral interior nave
6. a Russian nesting doll, opened, all layers visible
7. an avocado halved, pit centered
8. a desktop PC tower with side panel removed, motherboard and GPU inside
9. a Roman amphitheater, ruined, arcade level visible
10. a beehive cross-sectioned, honeycomb structure visible

Open each in Blender; manually score 0/1/2 (none / some / meaningful interior).

Pass: sum ≥ 12/20 with ≥ 4 prompts scoring 2.

Adapt paths:

- 8–11: restrict launch scope to "naturally cross-sectionable" categories (anatomy, food). Drop mechanical from launch demos.
- < 8: dissection thesis dies. Pivot to PBR-relightable framing (object viewer + "drop into your photo" demo). Re-plan.

Spike B — HunyuanWorld walkability (Day 2)

Generate 3 scenes via HunyuanWorld on H100. Render a fly-through video along a hand-defined Bezier spline.

Pass criteria (all three must hold):

1. Scenes generated without errors in < 8 min each.
2. Fly-through has no obvious tearing, holes, or aliased garbage when the camera moves.
3. Geometry coherent enough that pedestals can be placed at sensible positions on a floor plane.

Adapt paths:

- One criterion fails: try one prompt-engineering iteration to fix it (cleaner prompts, different seed). If still fails on retry: drop to Approach C (stock rooms only).
- Two+ fail: drop to Approach C with no retry. Update spec; ship without HunyuanWorld.

Spike C — three.js capped clipping on TRELLIS.2 GLB (Day 3)

Use the best-scoring GLB from Spike A (likely the watch or pomegranate). Build a minimal three.js scene that:

1. Loads the GLB with `GLTFLoader` and renders PBR materials under a fixed HDRI environment.
2. Implements a movable clipping plane on a slider.
3. Implements **capped** cross-section via stencil buffer.
4. Maintains 60fps on a mid-range laptop GPU (e.g., M2 MacBook Air, RTX 3060 mobile).

Pass: capped clipping works without obvious visual bugs (Z-fighting, wrong cap color, stencil leaks).

Adapt paths:

- Clipping works, capping doesn't: ship v1 *without* capping. Mark v2.
- Clipping itself breaks on TRELLIS.2 mesh topology (multi-mesh hierarchy, non-manifold geometry): swap to exploded-view-only dissection — less impressive, still novel.
- Both fail: re-evaluate launch — likely indefensible without dissection. Hold launch.

Spike deliverable

`SPIKE_RESULT.md` committed to repo, containing:

- Per-spike pass/fail + score breakdown.
- One representative output (GLB, scene flythrough video, screen recording of capped clipping).
- Decision: green-light spec unchanged, scope-trim, or pivot.

If green-light, week 2 begins immediately.

---

§11 — Six-week build plan

(Detailed task-level plan goes in `docs/superpowers/plans/` via `superpowers:writing-plans` after this spec is approved.)

Week	Theme	Output
1	Spike + project init	SPIKE_RESULT.md; repo scaffolded; empty stack returns 501 from ML routes.
2	Backend + ML plumbing	FastAPI exhibits API. Modal functions for scene + object gen working end-to-end on test prompts. Orchestrator producing valid manifests.
3	Viewer foundation	three.js viewer with scene loading, pedestal placement, rails locomotion (no focus mode, no dissection yet). End-to-end real exhibit visible.
4	Dissection — the moat	Focus mode, clipping plane, capped cross-section, exploded view. The technical centerpiece.
5	Curation, polish, content	Three launch exhibits hand-tuned to perfection. Landing page. Curated gallery. Open-prompt UX. Error states. Cross-browser test.
6	Launch	Three demo videos. Domain DNS. Production deploy. Soft-launch to friends. Public launch — tweet, Show HN, /r/threejs, /r/blender.

Scope-cut order (if weeks slip): drop Botany exhibit first (ship in week 7); then drop capped cross-section (ship plain clipping); then drop HunyuanWorld (ship Approach C only).

---

§12 — Error handling and degradation

Failure	Detection	Handling	User-facing
Orchestrator: Claude API down	exception	retry once with backoff	"Try again" + 503
Orchestrator: invalid JSON or schema fail	Pydantic	retry once with validation error in prompt	second fail → 502, "Could not plan this exhibit"
Orchestrator: refused prompt	`refused: true` in output	400 with reason	"This prompt couldn't be exhibited: <reason>"
Scene gen: HunyuanWorld timeout / garbage	output validation (file present, geom bounds, test-render)	re-orchestrate with `force_generator=stock`	SSE `scene_fallback` event; small footer flag
Object gen: one object fails	per-object output check	append to `manifest.errors`, continue	missing pedestal shows "Could not generate this exhibit item" placard
Frontend: GLB load fails	three.js error	pedestal shows ghost outline	hover tooltip explains
Frontend: WebGL context lost	event listener	"Reload page" overlay	—
Network drop mid-load	retry w/ backoff	—	spinner + retry button after 10s

User navigates away mid-gen	backend continues	manifest writes regardless	next visit picks up result

Stages that aren't implemented yet (M0 of build) return **501** with `{ stage, detail }`. Faking outputs is forbidden — wiring bugs hide behind mocks.

---

§13 — Testing strategy

Portfolio mode: testing is light, but disciplined where it matters most (the manifest contract).

- **Manifest schema:** Pydantic validates every orchestrator output. `test_manifest_schema.py` with 5 hand-written manifests (one per launch exhibit + edge cases: single object, no pedestals, stock fallback) plus 5 invalid manifests that must fail validation.
- **Orchestrator snapshot:** ~10 fixed prompts; commit accepted outputs as fixtures (`tests/fixtures/orchestrator/<slug>.json`); re-run when system prompt changes. Manual eyeball review for sanity; no automatic equality check (LLM nondeterminism).
- **Spike outputs:** SPIKE_RESULT.md is the week-1 test report.
- **Frontend:** no unit tests in v1. Manual smoke + Playwright script for the three launch exhibits before each deploy.
- **End-to-end smoke:** `make smoke` hits `POST /api/exhibits` with `"a Victorian study with a pocket watch"` , polls `/api/exhibits/{id}` until ready, asserts manifest has `stage=ready` , ≥3 objects, all GLB URLs return 200.

No code coverage target. No TDD. Move fast, demo well.

---

§14 — Risks

#	Risk	Likelihood	Impact	Mitigation	Trigger to revisit
1	TRELLIS.2 interiors are marketing more than reality	M	High (kills dissection thesis)	Spike A	Week 1
2	HunyuanWorld scene quality is poor under camera motion	M	High (breaks immersion)	Spike B + Approach C fallback	Week 1
3	Capped clipping shader doesn't work on TRELLIS.2 mesh topology	M	M (dissection ships hollow)	Spike C	Week 1
4	Per-exhibit cost climbs above $1 due to H100 time	L	M	Default 512^3 tier; 1024^3 only for curated; hard cap $200/mo Modal budget	Week 4
5	Orchestrator produces inconsistent layouts	M	M	Hand-tune layout templates + few-shot	Week 5

	(objects clip into walls, pedestals overlap)			examples; validate positions against scene bbox	
6	Two-minute generation time kills casual visitors	Certain	M	Curated gallery on landing page so first interaction is instant	Pre-launch
7	Modal cold starts make first request painful (5+ min)	M	M	Keep Modal functions warm during demo windows; honest UI copy about latency	Week 6
8	Demo goes viral, Modal bill explodes	L	M	Daily spend cap; per-IP rate limit (5 exhibits/day)	Pre-launch
9	Browser memory limits on weak laptops (large GLB + 4K env)	M	M	Compress GLBs (Draco / Meshopt); test on M1 MacBook Air	Week 5
10	TRELLIS.2 produces objects with wrong scale (giant watch, tiny cathedral)	H	M	Normalize each GLB to unit-bbox post-load; manifest `scale` field handles per-object correction	Week 4

---

§15 — Out of scope, v2 hooks

Hard out of scope for v1

- Free WASD walking with collision.
- Object physics / pick-up / throw.
- Audio.
- Multi-user / shared sessions.
- Mobile viewer.
- Editing primitives.
- Auth, accounts, payments.
- Production-grade content moderation (prompt-level Claude filter is the only line in v1).

Hooks left for future evolution

- The pluggable Modal-function pattern allows swapping HunyuanWorld for a future scene model without re-architecture.
- The orchestrator system prompt is versioned — quality improves via prompt-engineering without touching code.
- Manifest schema is versioned (`schema_version: "v4.1"`) — additive fields are forward-compatible.
- Stock-room library can grow indefinitely; orchestrator picks by metadata.
- Dissection panel is a single React component — adding more interaction primitives (label hover, info-on-hover, etc.) is local change.

Things we might add in v2 (not designed here)

- Curator mode: a user can pin/regenerate objects, reposition pedestals, lock and re-share.
- Educational packaging: exhibit "tour" with narration text per station.
- Embedding: `<iframe>` snippet for blog posts.
- Multi-room exhibits: chain N rooms together.
- WebXR viewing in Quest/Vision Pro.
- Audio: AI-generated ambient sound per scene.

---

§16 — Open questions

Resolved unless noted.

- **Domain name / branding** — TBD; deferred to week 5. "Museum" stays as the *category description* in copy; the brand can be anything.
- **Content moderation** — v1 is Claude refusal in the orchestrator only. Output-level moderation is v2.
- **Storage growth** — Modal Volume is fine for v1 traffic; ~500MB per exhibit × ~100 exhibits/week = 50GB/month. Modal free tier is 50GB — we'll watch and migrate to R2 if needed.
- **Analytics** — Plausible (privacy-friendly, free for low traffic).
- **License of generated exhibits** — TBD; default copy: "Shareable; not for commercial reuse without permission."
- **Curated-room sourcing** — 10 rooms by week 1 spike close (parallel work).
- **HunyuanWorld output format** — Spike B will determine whether it emits GLB, PLY, or a hybrid format. Conversion adapter if needed.

---

§17 — Acceptance for this spec

This spec is accepted when:

- All sections (§3–§13) have been reviewed and approved by the project owner.
- Any feedback has been folded in inline.
- The owner explicitly says: *"approved, write the implementation plan."*

Next step after acceptance: invoke `superpowers:writing-plans` to convert this into a task-level implementation plan saved to `docs/superpowers/plans/`.